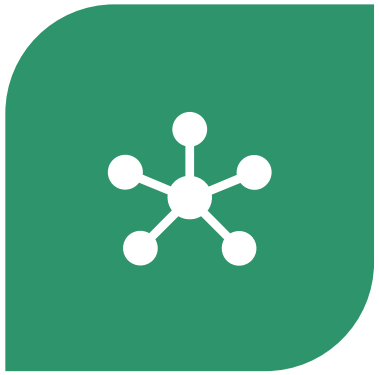
CSS3 & Responsive Design

Focus: Theory, standards, architecture, examples

CSS3 & Responsive Web Design

MODERN LAYOUT SYSTEMS

STANDARDS



DESIGN PARADIGMS

Learning Objectives

After this lecture, students should be able to:

- Understand the CSS3 specification model
- Explain modern layout systems (Flexbox, Grid)
- Analyze responsive design strategies
- Choose appropriate units, breakpoints, and approaches
- Critically evaluate responsive architectures

CSS3 in the Context of Web Standards

CSS3 is **not a single monolithic specification**

It is a **collection of independent modules**

Modules evolve at different speeds:

- Selectors
- Backgrounds & Borders
- Values & Units
- Flexbox
- Grid Layout

Maintained by **W3C**

Why CSS3 Matters

- Separation of concerns (HTML vs CSS)
- Performance (less JavaScript-driven layout)
- Accessibility & maintainability
- Device diversity (screens, orientations, resolutions)

CSS Rendering Model

- DOM tree + CSSOM → Render Tree
- Layout → Paint → Composite
- CSS impacts:
- Reflow (layout)
- Repaint (visual)
- Responsive design directly affects layout calculations

Modern CSS Selectors

- Reduces dependency on extra classes
- Improves maintainability

```css
article > section:not(.featured) {
  margin-bottom: 2rem;
}

li:nth-child(2n + 1) {
  background-color: #f5f5f5;
}
```

Color Models in CSS3

- Supported color systems:
- RGB / RGBA
- HSL / HSLA
- HEX
- currentColor

📌 HSL is closer to human color perception

```css
button {
  background-color: hsla(210, 80%, 50%, 0.9);
  color: white;
}
```

Gradients as First-Class Citizens

No images required:

Advantages:

- Resolution-independent
- Smaller payload
- GPU-accelerated

```css
header {
  background: linear-gradient(135deg, #667eea, #764ba2);
}
```

Typography in CSS3

- @font-face
- Web-safe vs custom fonts
- Font loading strategies

📌 font-display: swap improves perceived performance

```css
@font-face {
  font-family: "Inter";
  src: url("Inter.woff2") format("woff2");
  font-display: swap;
}
```

The Problem of Layout (Historical Context)

- Legacy approaches:
- Tables ❌
- Floats ❌
- Inline-block ❌
- Problems:
- Poor semantics
- Complex hacks
- Fragile responsiveness

Flexbox: One-Dimensional Layout Model

Designed for:

- Alignment
- Distribution
- Directional layouts

Key concept:

- Layout along **one axis at a time**

Flexbox Core Properties

- Main axis vs cross axis

Automatic space distribution
Intrinsic responsiveness

```css
.container {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
}
```

Flexbox Sizing Algorithm

- Flex-basis
- Grow and shrink factors
- Content-based sizing

📌 Understanding the algorithm is critical for complex layouts

```css
.item {
  flex: 1 1 300px;
}
```

Responsive Web Design: Definition

Coined by **Ethan Marcotte**

Three pillars:

- Fluid grids
- Flexible media
- Media queries

Device Diversity Challenge

- Phones, tablets, desktops
- Foldables
- TVs
- High DPI displays
- Orientation changes
- 📌 Fixed layouts are no longer viable

Responsive vs Adaptive

Responsive	Adaptive
Fluid	Breakpoint-based
Continuous	Discrete layouts
CSS-driven	Often device-driven

Modern systems often **combine both**

Units of Measurement

Absolute vs relative units:

- %
- em, rem
- vw, vh
- clamp()

📌 clamp() reduces media query dependency

```css
h1 {
  font-size: clamp(1.5rem, 4vw, 3rem);
}
```

Media Queries

Core mechanism of adaptation:

Media features:

- width / height
- orientation
- resolution
- prefers-color-scheme

```css
@media (max-width: 768px) {
  nav {
    flex-direction: column;
  }
}
```

Mobile-First Strategy

Principle:

- Start with the smallest viewport

Advantages:

- Performance
- Cleaner CSS
- Progressive enhancement

```css
body {
  font-size: 1rem;
}

@media (min-width: 1024px) {
  body {
    font-size: 1.125rem;
  }
}
```

CSS Grid Layout

Two-dimensional layout system

Best for:

- Page-level layouts
- Complex grids

Core Concepts

- Grid Container
- Grid Items

```css
.container {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
  grid-template-rows: auto 200px;
  gap: 20px;
}
```

•grid-template-columns defines three columns, the first and last flexible (1fr), the middle twice as wide (2fr)

•grid-template-rows defines two rows: the first adjusts automatically to content, the second is 200px

•gap defines spacing between rows and columns

Advantages of CSS Grid

•**Two-dimensional control** — rows and columns simultaneously
•**Precise placement** — explicit positioning of items
•**Flexible sizing** — using `fr`, `auto`, and `minmax()` functions
•**Reduced markup complexity** — fewer wrapper elements than with Flexbox alone
•**Responsive-friendly** — works with media queries, relative units, and container queries

Grid vs Flexbox

Feature	Flexbox	Grid
Direction	One-dimensional	Two-dimensional
Best for	Components, menus, toolbars	Full-page layouts, dashboards
Item sizing	Flexible along main axis	Flexible along rows and columns
Complexity	Simple	Complex, precise

Criteria	Flexbox	Grid
Layout type	Linear, one-dimensional	Complex, two-dimensional
Components	Nav bars, cards, buttons	Dashboards, forms, gallery layouts
Control	Alignment, spacing, wrapping	Explicit placement, spanning multiple areas
Responsiveness	Flow items automatically	Restructure layout with breakpoints

In modern CSS, Flexbox and Grid are **complementary**.

Flexbox handles alignment inside a component; Grid organizes components across the page.

Performance Considerations

1. Layout Complexity

- Deeply nested Flexbox/Grid slows rendering
- Use Grid for overall layout, Flex for components
- Minimize excessive `auto` sizing

2. Media Queries & CSS Overhead

- Multiple/overlapping queries increase CSS processing
- Consolidate queries
- Use mobile-first approach (`min-width`)
- Prefer relative units (`fr`, `%`, `vw`, `vh`)

3. Images & Media

- Use `srcset` and `sizes` for responsive images
- Optimize image resolution for device
- Reduce page load on mobile

4. Animations & Transitions

- Use `transform` and `opacity` (GPU-accelerated)
- Avoid animating `width`, `height`, or layout properties

5. Minimize Repaints & Reflows

- Batch DOM updates
- Use `contain: layout;` where possible
- Avoid `!important` overrides

6. Accessibility & Performance

- Ensure layouts work on slow/low-power devices
- Maintain readability and functionality
- Combine responsive units and media queries efficiently

Accessibility & Responsiveness

1. Text & Typography

- Use relative units (`em`, `rem`, `%`)
- Ensure readable font size across devices
- Support user-adjusted font settings

2. Color & Contrast

- Maintain sufficient contrast for readability
- Support light/dark modes (`prefers-color-scheme`)
- Avoid color-only cues for critical information

3. Navigation & Interaction

- Ensure touch targets are large enough for mobile
- Support keyboard navigation
- Avoid hover-only interactions

4. Responsive Media

- Images and videos scale correctly
- Use `alt` attributes for screen readers
- Avoid content overflow on small screens

5. Motion & Animation

- Respect `prefers-reduced-motion`
- Avoid excessive or distracting animations

6. Testing & Validation

- Test across multiple devices and screen sizes
- Use accessibility auditing tools (Lighthouse, Axe)
- Combine responsive and accessible design principles

```css
@media (prefers-reduced-motion: reduce) {
  * {
    animation: none;
  }
}
```

Common Architectural Mistakes

1. Over-Nesting Layouts

- Excessive div wrappers
- Complex Flex/Grid hierarchies
- Hard to maintain and slows rendering

2. Fixed Units Everywhere

- Using only `px` for widths, fonts, and spacing
- Breaks responsiveness on small/large screens

3. Ignoring Mobile-First Principles

- Designing for desktop first, then shrinking
- Leads to bloated CSS and poor mobile performance

4. Overusing Media Queries

- Too many breakpoints or overlapping queries
- CSS becomes hard to maintain and less efficient

5. Non-Responsive Media

- Images, videos, and components that don't scale
- Causes overflow, horizontal scrolling, and layout breakage

6. Neglecting Performance & Accessibility

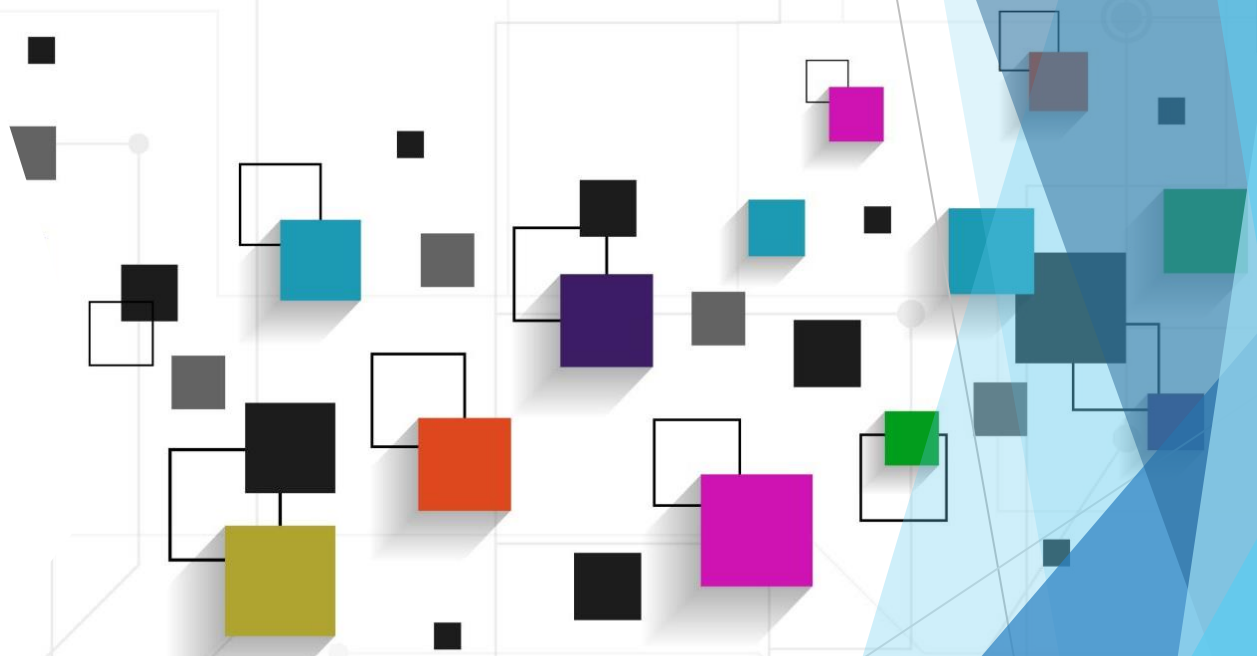
- Heavy layouts and animations
- Poor contrast, small touch targets, or inaccessible navigation

7. Relying on Absolute Positioning Too Much

- Limits flexibility of layouts
- Breaks under dynamic content or viewport changes

Summary

- CSS3 is modular and evolving
- Flexbox and Grid are foundational
- Responsive design is a **system**, not a feature
- Modern CSS reduces complexity and improves resilience



Recommended Reading

- MDN Web Docs (CSS)
- CSS-Tricks Almanac
- W3C CSS Specifications
- “Responsive Web Design” — Ethan Marcotte

1. Which statement best describes CSS3?

A. A single monolithic specification replacing CSS2
B. A collection of independent, modular specifications
C. A browser-specific styling language
D. A JavaScript-based layout system

2. Which organization is responsible for CSS standardization?

A. WHATWG
B. ECMA
C. W3C
D. ISO

3. Which stages are part of the browser rendering pipeline?

A. DOM construction
B. CSSOM construction
C. Layout
D. Paint and compositing

Multiple choice + conceptual questions

4. Which CSS properties are most likely to trigger a layout (reflow)?

A. color
B. margin
C. width
D. opacity

5. What is the main advantage of advanced CSS selectors?

A. They improve JavaScript execution speed
B. They reduce the need for additional markup and classes
C. They replace HTML semantics
D. They eliminate the CSS cascade

6. Why is the HSL color model often preferred in design systems?

A. It is more performant than RGB
B. It matches human perception of color adjustments
C. It uses fewer bytes
D. It is required for gradients

7. Which statement about CSS gradients is correct?

A. They require raster images
B. They are resolution-dependent
C. They are rendered by the browser and scale infinitely
D. They cannot be animated

8. What problem did Flexbox primarily solve?

A. Two-dimensional page layouts
B. Data visualization
C. Alignment and space distribution in one dimension
D. Server-side rendering

9. In Flexbox, what defines the primary layout direction?

A. justify-content
B. align-items
C. flex-direction
D. flex-wrap

10. Which components define Responsive Web Design according to Ethan Marcotte?

A. Fluid grids
B. Flexible media
C. Media queries
D. Server-side device detection

11. What is the main difference between responsive and adaptive design?

A. Responsive uses JavaScript, adaptive uses CSS
B. Responsive is fluid, adaptive relies on fixed breakpoints
C. Adaptive works only on mobile devices
D. Responsive requires multiple HTML files

12. Which CSS units are relative by nature?

A. px
B. %
C. rem
D. vw

13. What is the main advantage of the clamp() function?

A. It eliminates the need for JavaScript
B. It locks font sizes to fixed values
C. It enables fluid scaling with defined limits
D. It replaces media queries entirely

14. Which statement about CSS Grid is correct?

A. It is limited to one-dimensional layouts
B. It is best suited for component-level alignment
C. It provides two-dimensional layout control
D. It replaces Flexbox in modern CSS

15. Which practices improve responsive accessibility?

A. Supporting reduced motion preferences
B. Allowing font scaling
C. Relying exclusively on pixel units
D. Adapting contrast for different conditions

Answer Key

1. B
2. C
3. A, B, C, D
4. B, C
5. B
6. B
7. C
8. C
9. C
10. A, B, C
11. B
12. B, C, D
13. C
14. C
15. A, B, D

Thank you